CSP 一维数组、二维数组、字符串专项练习题 详细解答

一、选择题（答案 + 解析）

1. **答案：B**

   解析：C++ 中数组**部分初始化**时，未赋值的元素会自动初始化为**0**。

2. **答案：B**

   解析：数组 `a[3]` 的合法下标为 **0、1、2**，访问 `a[3]` 属于**数组越界**，会导致程序异常。

3. **答案：A**

   解析：C++ 二维数组定义格式为 `数据类型 数组名[行数][列数]`，`int b[2][3]` 表示**2 行 3 列**。

4. **答案：A**

   解析：`strlen` 函数计算字符串长度时，**不计入结束符** `\0`，`"abc"` 有效字符为 3 个，长度是 3。

5. **答案：A**

   解析：`sum = 10+20+30+40 = 100`。

6. **答案：B**

   解析：用字符串常量初始化字符数组时，系统会**自动在末尾添加结束符** `\0`。

7. **答案：C**

   解析：二维数组下标从 0 开始，`c[1][1]` 表示**第 2 行第 2 列**，元素值为 4。

8. **答案：B**

   解析：字符串比较按**ASCII 码逐位对比**，`abc` 第三位是 `c`，`abd` 第三位是 `d`，`c < d`，故 `s1 < s2`。

9. **答案：B**

   解析：数组下标从 0 开始，`arr[0]=2、arr[1]=5、arr[2]=8`，元素 8 的下标是 2。

10. **答案：C**

    解析：字符串 `CSP2026` 中数字为 `2、0、2、6`，共**4 个**。

二、程序设计题（思路 + 代码 + 解析）

1. 数组求和

题目：输入 n 个整数存入一维数组，计算数组所有元素的和并输出。

思路：定义数组→输入 n→循环录入元素→遍历累加求和→输出结果。

```
#include <iostream>
using namespace std;
int main() {
    int n, sum = 0;
    int arr[1000]; // 定义足够大的数组
    cin >> n;
    // 输入数组元素
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
    // 累加求和
    for (int i = 0; i < n; i++) {
        sum += arr[i];
    }
    cout << sum << endl;
    return 0;
}
```

解析：通过两次循环完成输入与求和，核心是**一维数组遍历 + 累加器。**

---

2. 数组最值

题目：输入 n 个整数存入一维数组，输出数组中的最大值和最小值。

思路：初始化 max/min 为第一个元素→遍历数组逐一比较→更新最值→输出。

```
#include <iostream>
using namespace std;
int main() {
    int n;
    int arr[1000];
    cin >> n;
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
    int max_val = arr[0];
    int min_val = arr[0];
    // 遍历找最值
    for (int i = 1; i < n; i++) {
        if (arr[i] > max_val) max_val = arr[i];
        if (arr[i] < min_val) min_val = arr[i];
    }
    cout << "最大值：" << max_val << endl;
```

```
    cout << "最小值：" << min_val << endl;
    return 0;
}
```

解析：以第一个元素为基准，遍历对比更新，是数组找最值的标准写法。

---

3. 数组逆序

题目：输入 n 个整数存入一维数组，将数组元素逆序后输出。

思路：双指针交换首尾元素→遍历完成逆序→输出数组。

```
#include <iostream>
using namespace std;
int main() {
    int n;
    int arr[1000];
    cin >> n;
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
    // 逆序交换
    for (int i = 0; i < n / 2; i++) {
        int temp = arr[i];
        arr[i] = arr[n - 1 - i];
        arr[n - 1 - i] = temp;
    }
    // 输出逆序数组
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    return 0;
}
```

解析：只需循环 `n/2` 次，交换 `i` 和 `n-1-i` 位置的元素即可完成逆序。

---

4. 元素次数统计

题目：输入 n 个整数和一个目标数，统计目标数在数组中出现的次数。

思路：定义计数器→遍历数组→匹配目标数则计数 + 1→输出计数。

```
#include <iostream>
using namespace std;
int main() {
    int n, target, count = 0;
    int arr[1000];
    cin >> n;
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
```

```
    }
    cin >> target; // 输入目标数
    // 统计次数
    for (int i = 0; i < n; i++) {
        if (arr[i] == target) count++;
    }
    cout << count << endl;
    return 0;
}
```

解析：核心是**数组遍历 + 分支条件计数**，适用于各类元素统计场景。

---

5. 3×3 矩阵求和

题目：输入一个 3 行 3 列的二维数组，计算所有元素的总和并输出。

思路：定义 3×3 二维数组→双层循环输入→双层循环累加→输出和。

```
#include <iostream>
using namespace std;
int main() {
    int arr[3][3];
    int sum = 0;
    // 输入矩阵
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            cin >> arr[i][j];
        }
    }
    // 累加求和
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            sum += arr[i][j];
        }
    }
    cout << sum << endl;
    return 0;
}
```

解析：二维数组必须用**双层 for 循环**遍历，外层控行、内层控列。

---

6. 矩阵转置

题目：输入一个 2 行 3 列的二维数组，将其转置为 3 行 2 列后输出。

思路：原数组`arr[2][3]`→转置后数组`res[3][2]`→赋值`res[i][j] = arr[j][i]`→输出。

```
#include <iostream>
using namespace std;
int main() {
```

```
    int arr[2][3]; // 原数组2行3列
    int res[3][2]; // 转置后3行2列
    // 输入原矩阵
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 3; j++) {
            cin >> arr[i][j];
        }
    }
    // 矩阵转置
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 2; j++) {
            res[i][j] = arr[j][i];
        }
    }
    // 输出转置矩阵
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 2; j++) {
            cout << res[i][j] << " ";
        }
        cout << endl;
    }
    return 0;
}
```

解析：矩阵转置核心是**行列下标互换**，固定遍历原数组行列，赋值给新数组。